

20 | OPA：为 Kubernetes 生态系统构建安全堡垒

潘野 · 云原生基础架构实战课



你好，我是潘野。

前面的课程里，我们着重讲解了云原生中的加密管理的方法与实现手段。

而在现代云原生生态系统中，Kubernetes 已成为容器编排的事实标准。但随着 Kubernetes 集群的规模和复杂性增加，其集群本身安全性管理也面临着越来越多的挑战。原来的安全策略已经逐渐跟不上新的形势，特别是在灵活性和细粒度控制方面。

今天我们将通过一个具体场景来探讨 Kubernetes 安全策略的不足，学习如何通过开放策略代理（OPA）弥补这些不足，并带你展望未来 Kubernetes 中条件表达式语言（CEL）的发展计划。

Kubernetes 安全性：从 PSP 到 OPA 的转变

我们先来看一个场景。

公司里运行着几个多租户 Kubernetes 集群，其中包括金融、零售、支付等等多个业务的应用程序，这些业务对安全性和数据隔离有着严格的要求。一般情况下，我们会将不同业务的程序放在不同的 namespace 中，配合 Kubernetes 的原生 Role-Based Access Control (RBAC) 来管理访问控制，但是这并不够用。

比如说，业务 A 在 deployment 中定义了使用 privilege 的权限，这个 deployment 启动之后的容器就有能力看到宿主机上的整个磁盘目录，这就增加了其他业务被攻击的风险。

所以 Kubernetes 中设立了一个安全机制，叫做 Pod Security Policies (PSP)。这是一种集群级别的安全机制，用于控制 Pod 的安全规范。PSP 旨在为 Pod 运行时提供安全保障，通过定义一系列条件来限制 Pod 可以使用的资源和权限。

PSP 可以做哪些安全策略呢？主要有如下几个方面。

1. 限制容器是否可以运行在特权模式下。
2. 控制容器是否可以挂载文件系统，以及挂载的类型。
3. 限制容器使用的网络模式，例如禁用 hostNetwork，从而防止容器访问主机网络。
4. 指定 Pod 中容器运行的用户和用户组。这可以防止容器以 root 用户运行，增加安全性。

PSP 提供了一系列强大的安全功能，但在实际使用中也暴露出一些明显的不足，最大的不足是它只能控制 pod，缺乏更细粒度的控制。比如不能基于单个用户或具体应用的差异化来配置安全策略，也不能控制其他的资源。

所以从 Kubernetes 1.21 版本开始，PSP 被标记为弃用，并在 1.25 版本中被移除。这迫使我们寻找其他的安全替代方案，那么此时社区推荐的方案是开放策略代理（Open Policy Agent，后面我们简称 OPA）。

如何理解 OPA？

你从网络上搜索到的关于 OPA 的介绍，大多都是这样的。

开放策略代理（Open Policy Agent）是一个开源的、通用的策略引擎，OPA 通过提供一个高级的策略语言—Rego（用于表达策略和查询数据）—让开发人员能够定义和执行跨微服务、Kubernetes、CI/CD 管道、API 网关等策略。

看到这个介绍，恐怕你会越看越迷惑。让我换个方式带你理解就会容易很多。通俗来说，OPA 就像框架一样，提供了接口、语法工具等，而具体的策略逻辑和需求，则需要你自己根据自己的应用场景来实现。

所以我们要将 OPA 的功能扩展到 Kubernetes 环境中，还需要一个真正的软件实现，这里就是 OPA 的 [Gatekeeper](#)。

Gatekeeper 是专为强化 Kubernetes 的安全性而设计的。它通过实施自定义策略来控制和管理对 Kubernetes API 的访问。有了 Gatekeeper，Kubernetes 管理员就可以定义细粒度的、声明式的策略。这些策略在资源创建或更新时自动执行，保证集群状态符合组织的合规性和安全标准。

OPA 的三大优势

那么具体来说，OPA 究竟在哪几个方面比 PSP 表现更好呢？主要是三个方面，我们依次看看。

第一，**支持细粒度控制**。OPA 的 Rego 语法允许用户编写非常细粒度和复杂的逻辑，不仅限于安全设置，还可以涵盖访问控制、数据过滤等多种用途。不仅可以对 Pod 进行管理，也可以对其他资源进行管理。

第二，**支持动态策略更新**。OPA 支持动态策略加载和更新，不需要重启服务或组件。这让管理大规模环境中的策略变得更加灵活和高效。

举个例子，如果我们想阻止非认证的容器镜像部署，以往的做法是在 API Server 的 Admission Webhook 中做拦截。但是这一步在首次部署的时候，需要更改 API Server 配置并重启 API Server，这会带来一定的风险。而使用 OPA 的方式，无需重启 API Server，配置即刻生效，这样就避免了繁琐操作。

第三是**支持审计和追踪功能**。Gatekeeper 会定期扫描 Kubernetes 集群中的所有资源，以验证它们是否符合所有激活的策略（Constraints）。我们也可以通过 Pub/Sub 模式导出审计结果。

Gatekeeper 的配置和使用

前面我们了解了 OPA 和 Gatekeeper 的各种优势，那我们怎么使用它们来防护集群呢？

首先，我们需要安装 Gatekeeper，直接运行下面这条命令即可，等待所有的 pod 都完成启动。

 复制代码

```
1 kubectl apply -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper/v
```

也可以使用 helm 的模式安装，命令如下：

 复制代码

```
1 helm repo add gatekeeper https://open-policy-agent.github.io/gatekeeper/charts
2 helm install gatekeeper/gatekeeper --name-template=gatekeeper --namespace gatekee
```

Gatekeeper 有两个重要的基本概念需要你关注。

第一个概念是 ConstraintTemplates，它定义了可以在集群中实施的策略类型。

一个 ConstraintTemplate 主要包含以下几个部分。

CRD 规范：定义了将由该模板创建的 CRD 的结构，包括其名称和属性。

Targets：指定该模板将应用于哪些 Kubernetes API 对象（如 Pod、Service 等）。

Rego Policies：使用 Rego 语言编写的实际策略逻辑，定义了资源必须满足的条件。

第二个概念是 Constraints。Constraints 是实际应用于集群资源的对象，这些对象基于 ConstraintTemplates 来定义。ConstraintTemplates 提供了策略的模板，而 Constraints 则是这些策略的具体实现，指定了策略应用的具体条件和目标。

那么接下来我们就结合一个案例，看看如何写出一些用于集群防护的策略。

案例：禁止用户使用 gp3 类型的 Storage Class

首先，我们创建一个 ConstraintTemplates。

 复制代码

```
1 apiVersion: templates.gatekeeper.sh/v1
2 kind: ConstraintTemplate
3 metadata:
4   annotations:
5     description: Denied creation of PVCs backed by Rook.
6   name: denygp3
7 spec:
8   crd:
9     spec:
10      names:
11        kind: DenyGP3
12      validation:
13        legacySchema: true
14   targets:
15   - rego: |
16       package gp3
17       violation[{"msg": msg}] {
18         sc := input.review.object.spec.storageClassName
19         contains(sc, "gp3")
20         msg := "gp3 persistent volumes are not allowed."
21       }
22   target: admission.k8s.gatekeeper.sh
```

对照上面讲的 ConstraintTemplates 的几个部分，我们简单了解一下这个配置的重点。

首先是 CRD 规范，这里我们定义了一个叫做 DenyGP3 的 API 资源。然后在 Targets 中，用 rego 的语法写了一段规则。在这个规则下如果提交的 PVC spec 中的 storageClassName 包含 gp3，命令行就会弹出“gp3 persistent volumes are not allowed”这一提示。

接下来我们需要创建 Constraints。

Constraints 的作用是指定策略应用的具体条件和目标，这里应用的目标是配置文件最后一行的 PersistentVolumeClaim，而作用的条件参数是 `enforcementAction`，它这里的值是 `deny`，也就是发现匹配了 `DenyGP3` 规则的请求，默认会拒绝；但如果请求是 `kube-system` 中的，则放行。

 复制代码

```
1 apiVersion: v1
2 items:
3 - apiVersion: constraints.gatekeeper.sh/v1beta1
4   kind: DenyGP3
5   metadata:
6     name: deny-gp3
7   spec:
8     enforcementAction: deny
9     match:
10      excludedNamespaces:
11      - kube-system
12      kinds:
13      - apiGroups:
14        - ""
15      kinds:
16      - PersistentVolumeClaim
```

这样就定义了一个控制策略，我们来看看效果。

这里我们需要定义一个 PVC 的配置文件，如下所示。

 复制代码

```
1 # cat pvc.yaml
2 apiVersion: v1
3 kind: PersistentVolumeClaim
4 metadata:
5   name: myclaim
6 spec:
7   accessModes:
8     - ReadWriteOnce
9   volumeMode: Filesystem
10  resources:
```

```
11     requests:
12         storage: 8Gi
13     storageClassName: gp3
```

然后 apply 下这个 PVC 配置，验证一下 OPA Gatekeeper 是否会阻拦这个请求。从输出结果你可以看到 Gatekeeper 阻拦了这个请求，并且抛出了 `gp3 persistent volumes are not allowed` 的报错。

 复制代码

```
1 # kubectl -n default create -f pvc.yaml
2 Error from server ([deny-gp3] gp3 persistent volumes are not allowed.): error whe
```

前面我提到过，OPA 是一个开源且通用的策略引擎，适用于多种软件系统中的授权和策略决策。它支持为 Kubernetes、API 网关等提供控制策略。这里我再稍微提一下 OPA 的其他用途，比如，[OPA 可以集成到 Envoy 代理中](#)，实现细粒度的访问控制。

其基本原理是使用 Envoy 的 External Authorization filter 配置代理，以在请求处理流程中嵌入 OPA 调用，OPA 中定义的策略策略可以基于请求的各种属性（如 HTTP 头、路径、方法等）进行细粒度控制。

OPA 的缺点

尽管 Open Policy Agent (OPA) 提供了强大的灵活性和广泛的适用性，它也存在一些潜在的挑战和限制。

首先，OPA 的学习曲线陡峭。前面我们了解到，OPA 使用 Rego 语言来定义策略。而 Rego 是一种声明性语言，除了[官方文档之外](#)，可参考的资料不是很多。对于不熟悉声明性或逻辑编程范式的开发者来说，Rego 可能难以理解和掌握。

其次是性能问题。在高负载或复杂策略的情况下，OPA 可能会遇到性能瓶颈。由于每个请求都需要评估策略，这可能导致延迟增加，特别是在分布式系统中延迟会更为严重。所以我们需

要从 rego 语法优化、请求量的预先评估，以及使用缓存减少对 OPA 服务的请求次数这几方面来综合解决，从而减少总体延迟。

最后还有一点，OPA Gatekeeper 用来控制 Kubernetes 集群的安全策略，OPA envoy 用来控制流量层的安全策略，两者功能比较类似，但是两者的配置却无法统一。这增加了我们学习和使用它的成本。

CEL 的发展计划和前景

前面说到 rego 的语法学习成本比较高，同时 OPA 属于外挂 controller，可能存在性能问题。Kubernetes 官方社区在决定废弃 PSP 之后，也看到了 OPA 的这些不足，并致力于设计下一代的安全控制策略，这就是我们接下来要讲的 CEL。

CEL (Common Expression Language) 是一种轻量级的表达式语言，旨在简化数据查询和操作，以此应对越来越复杂的安全需求。Kubernetes 社区正在探索将 [CEL](#) 集成到不同的 Kubernetes 组件中，以提供更灵活的配置和策略决策能力。

CEL 可用于以下场景：

1. Admission Control: Kubernetes 的 Admission Controllers 负责在对象创建或更新时执行策略检查。通过使用 CEL，管理员可以编写更复杂的入场条件或验证逻辑，而无需编写和维护复杂的 Webhook 服务。
2. Custom Resource Definitions (CRDs): 在 CRDs 中，CEL 可用于定义更复杂的验证规则，这些规则超出了标准的基于类型的验证。
3. API Gateway: 在 API 网关中，CEL 可用于定义复杂的路由规则，如基于请求内容的动态路由决策。

我们来看一个例子，我们定义了一个校验规则，当创建 (create) 或者修改 (update) deployment 的时候，spec 中的 replicas 这个值需要大于等于 3。

 复制代码

```
1 apiVersion: admissionregistration.k8s.io/v1alpha1
2 kind: ValidatingAdmissionPolicy
```

```
3 metadata:
4   name: "deploy-replicas-policy"
5 spec:
6   matchConstraints:
7     resourceRules:
8       - apiGroups: ["apps"]
9         apiVersions: ["v1"]
10        operations: ["CREATE", "UPDATE"]
11        resources: ["deployments"]
12   validations:
13     - expression: "object.spec.replicas >= 3"
```

比较一下，用 CEL 编写这样的规则，是不是比 OPA Gatekeeper 更为简洁明了？

虽然 CEL 在 Kubernetes 中的应用还在开发阶段，但它的引入预示着更灵活和强大的策略定义方式。相信随着时间的推移，CEL 将在 Kubernetes 生态系统中扮演越来越重要的角色。

总结

我们今天深入探讨了云原生环境中，特别是在 Kubernetes 生态系统中安全管理的演变和挑战。

通过回顾传统的安全策略，如 Kubernetes 的 Role-Based Access Control (RBAC) 和 Pod Security Policies (PSP)，我们发现它们在应对现代化多租户集群环境中的局限性，特别是在灵活性和细粒度控制方面。

正是为了解决这样的问题，开放策略代理（OPA）应运而生，随之而来的还有 OPA 在 Kubernetes 中的实现——Gatekeeper。OPA 通过其策略语言 Rego 提供了一种灵活、细粒度的安全策略定义方式，使得管理员可以更精确地控制和管理集群资源。

相比于 PSP，OPA 支持更复杂的逻辑、动态策略更新以及细粒度的控制，虽然它也带来了一定的学习曲线和潜在的性能挑战，但优势依然很明显。

另外，我们了解了 Kubernetes 社区正在研究的下一代安全策略工具——条件表达式语言（CEL）。CEL 旨在提供一种更简洁、更强大的方式来表达策略和规则，预计它将进一步简化

策略的定义并提高执行效率。

思考题

请你用 Gatekeeper 定义一个策略，规则是容器启动的时候禁止使用 host network。

欢迎在评论区与我讨论。如果这一讲对你有启发，也欢迎分享给身边更多朋友。

AI智能总结

1. Kubernetes安全挑战：随着Kubernetes集群规模和复杂性的增加，传统安全策略已经跟不上新形势，特别是在灵活性和细粒度控制方面。
2. PSP的不足：Pod Security Policies (PSP)在实际使用中暴露出一些明显的不足，最大的不足是它只能控制pod，缺乏更细粒度的控制，不能基于单个用户或具体应用的差异化来配置安全策略。
3. OPA的优势：OPA相比PSP在支持细粒度控制、动态策略更新和审计追踪功能方面表现更好，允许用户编写非常细粒度和复杂的逻辑，支持动态策略加载和更新，以及定期审计和追踪功能。
4. Gatekeeper的配置和使用：Gatekeeper是专为强化Kubernetes安全性而设计的，通过实施自定义策略来控制和管理对Kubernetes API的访问，需要安装Gatekeeper并了解ConstraintTemplates和Constraints的概念。
5. OPA和Gatekeeper的实际应用：通过案例展示了如何使用OPA和Gatekeeper来实现细粒度的、声明式的策略，以确保集群状态符合组织的合规性和安全标准。
6. CEL的发展计划和前景：Kubernetes社区正在探索将CEL集成到不同的Kubernetes组件中，以提供更灵活的配置和策略决策能力，CEL可用于Admission Control、Custom Resource Definitions (CRDs)和API Gateway，预示着更灵活和强大的策略定义方式。
7. 总结：传统安全策略在应对现代多租户集群环境中存在局限性，OPA和Gatekeeper提供了更灵活、细粒度的安全策略定义方式，而CEL的引入预示着更灵活和强大的策略定义方式。
8. 思考题：Gatekeeper定义一个策略，规则是容器启动的时候禁止使用host network。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

精选留言

由作者筛选后的优质留言将会公开显示，欢迎踊跃留言。

